Day 3 - Oct 22: Conditions, CTEs, Window Functions

Swagat | Revision sheet | Open item: SQL 5-question check

1. KEY IDEAS

Python: Conditions

- if needs a colon at the end of the line AND indentation (4 spaces) below it. Missing indent = IndentationError, a syntax error: nothing runs, not even earlier lines.
- else has no condition. elif has its own condition. Both line up with the if, only their bodies are indented.
- if / elif / else chain: top to bottom, first True wins, the rest is skipped. A plain second if is a NEW independent test, so both can run.
- and: both True. or: at least one True. not: flips. Keywords are lowercase. Order: not, then and, then or. Chained: 60 <= marks < 80.
- Nested if: an else belongs to the if at the SAME indentation. Flat code (and) is often cleaner.
- Put the strictest condition first, and test boundary values (39, 40, 59, 60, 79, 80).
- Ties: use >= when the biggest value can repeat (largest of three). The last else is a silent assumption.

SQL: CTE and window functions

- CTE: WITH name AS (query). Lives for ONE statement only. Several CTEs: one WITH, separated by commas, NO comma after the last.
- Temp table lasts the whole session: CREATE TEMP TABLE t AS SELECT ... A CTE is not a saved table.
- Window function = function() OVER (...). Keeps every row and adds a column. GROUP BY collapses rows. Cannot be used in WHERE: compute it in a CTE, then filter outside.
- RANK 1,2,2,4 | DENSE_RANK 1,2,2,3 | ROW_NUMBER 1,2,3,4 (add a tie-breaker in ORDER BY).
- PARTITION BY restarts the calculation for each group. ORDER BY inside OVER sets the order used.
- LAG(col) OVER (ORDER BY ...) = value from the previous row, NULL for the first. Add PARTITION BY to keep groups separate.
- Running total: SUM(x) OVER (ORDER BY day). Per group: add PARTITION BY. Rows with equal ORDER BY value share a total unless you add ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.
- WHERE filters rows before grouping. HAVING filters groups after grouping.
- Order months by a real key such as strftime('%Y-%m', order_date), never by month names (text sorts Apr, Feb, Jan, Mar).

Git (theory only on this day)

- Repository, working folder, commit = saved snapshot. Practical was done on Oct 23.

2. EXAMPLES

```
age = 20
if age >= 18:
    print('adult')      # indented under the if
else:
    print('minor')

marks = 72
if marks >= 80: print('A')
elif marks >= 60: print('B')   # first True wins -> B
else: print('C')
```

```
WITH customer_revenue AS (
  SELECT o.customer_id, SUM(oi.quantity*p.price) AS revenue
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  JOIN products p ON p.product_id = oi.product_id
  GROUP BY o.customer_id
)
SELECT * FROM customer_revenue ORDER BY revenue DESC LIMIT 5;
```

```
SELECT Employee, Department, Salary,
       RANK() OVER (PARTITION BY Department ORDER BY Salary DESC) AS Salary_Rank
FROM Employees;
```

3. MY MISTAKES AND THE FIX

- Said only 'indentation' -> forgot the colon. Fix: colon + indent, always both.
- Thought a missing indent still checks the condition -> wrong. Python reads the whole file first; syntax errors stop everything.
- Two plain ifs, marks = 85: answered 'a' -> both A and B print, and strings are case-sensitive. Fix: plain if is independent; elif links to the test above.
- Wrote AND in capitals -> keywords are lowercase: and, or, not.
- not False: predicted 'prints nothing' -> it prints 'go out'. Fix: evaluate the condition fully, THEN decide if the body runs.
- Largest of three, input 5,5,3 gave 'C is largest' (wrong) -> strict > fails on ties. Fix: >=. Also test inputs that can FAIL, not only ones that pass by luck.
- CTE: thought SELECT * FROM cte works in the next statement -> error. A CTE exists for one statement only.

- Answered 'Rank?' to a numbering question -> if stuck, give a reasoned guess. RANK skips numbers after a tie.
- Could not write RANK() OVER (PARTITION BY ... ORDER BY ...) -> learn the fixed shape: two blanks, group column then ranking column.
- SQL practice: PARTION typo; window function placed after FROM; missing comma before the function; missing alias; extra comma before FROM; extra comma after GROUP BY; ORDER BY Salary on a table with no Salary column. Result: 1 of 3 right first try, all 3 right after fixes.
- Rule from these: read the table's columns first. Clause order: SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY. Once a table has an alias, use only the alias.

4. HOW I IMPROVED

- Last-three-values and elif questions: by the end I gave output + reason + fix together. Wrote the leap/else logic with strictest-first without being told.
- Recall of CTE vs temp table and PARTITION BY vs GROUP BY came out in my own words.

5. PRACTICE TO RETRY (no solutions)

- Grade from marks with Invalid for below 0 or above 100. Login check, nested and flat. Largest of three with a tie (5,5,3).
- SQL: top 5 customers by revenue with a CTE. Rank salaries inside each department. LAG for previous month. Customers with total above 200, sorted high to low.
- Weak spots to re-test: str + int TypeError, immutability, '' vs ' ', SUMIFS order, pivot source range, hidden spaces causing #N/A.

6. RECALL QUESTIONS (answer without notes)

- What two things mark an if block? What does a missing indent do?
- 85 marks, two plain ifs (80 and 60): what prints? Change one word so only A prints.
- What does or return for '' or 'guest'? Which runs first: and or or?
- Why can a CTE not be queried in the next statement? What replaces it?
- Why can RANK not go in WHERE? How do you filter on it?
- RANK vs DENSE_RANK vs ROW_NUMBER for 900, 800, 800, 700.
- WHERE vs HAVING.

7. OPEN ITEMS

- SQL 5-question check on Colab data (top 5 customers, best product per category, customers with 2+ orders, month-over-month growth %, running total by month): NOT DONE.
- Recall log: done (see README). Git practical moved to Oct 23.